

SuperBizAgent

企业级 Incident Response Agent 项目详解

从“看懂架构图”到“能讲清路由、升级、审批与恢复”

易懂增强版学习教程

面向秋招项目讲解与源码学习，完整描述单一 Durable Incident Runtime、Simple / Enterprise 双策略和共享安全底座。

依据当前代码、README、测试证据与配套流程图整理

版本：2026-08

阅读说明

这份教程把项目拆成入口、普通 Chat、知识入库、统一事故 Runtime、工具治理、状态持久化、企业策略、运行边界和面试表达。每章都先用大白话说明作用，再给出技术链路、代码位置和容易混淆的点。

先记住一句话

这是一个面向企业 OnCall / SRE 场景的事故响应 Agent 平台。普通 Chat 保持独立；所有正式事故请求进入同一个 AIOpsService，由确定性 Router 选择 Simple 或 Enterprise，并共享 PostgreSQL Checkpoint、Tool Gateway、审批、证据和审计。

建议阅读路线

- 第一次看项目：先读第 1、2、5、7、9 章，形成统一 Runtime 心智模型。
- 准备面试：重点读第 5、6、7、8、11、12 章，讲清自动路由、动态升级和审批恢复。
- 准备改代码：读第 4、10 章，再沿 API -> Service -> Graph Node -> Storage 进入源码。
- 排查运行问题：读第 9、11 章，确认 Milvus、PostgreSQL、MCP、Prometheus 和 Provider 边界。

全书目录

- 1. 一分钟看懂整个项目
- 2. 一次 HTTP 请求如何进入 FastAPI
- 3. 普通聊天：Agent + RAG + MCP
- 4. 文档如何进入 Milvus 知识库
- 5. 项目核心：单一 Durable Incident Runtime
- 6. Tool Gateway：为什么企业 Agent 必须管住工具
- 7. 路由、升级、审批、Checkpoint 与恢复
- 8. Enterprise 多智能体诊断策略
- 9. 启动、存储与运行边界
- 10. 代码目录应该按什么顺序读
- 11. 测试、调试和常见故障定位
- 12. 面试时怎么把项目讲清楚
- 附录 A. 关键术语速查
- 附录 B. 配套架构图索引与自测

1. 一分钟看懂整个项目

从业务视角看，系统是一个 FastAPI 接待台和两类处理域：普通 Chat / RAG 负责快速问答；Durable Incident Runtime 负责正式事故。事故 Runtime 内部不是两套服务，而是 Simple 与 Enterprise 两种诊断策略。

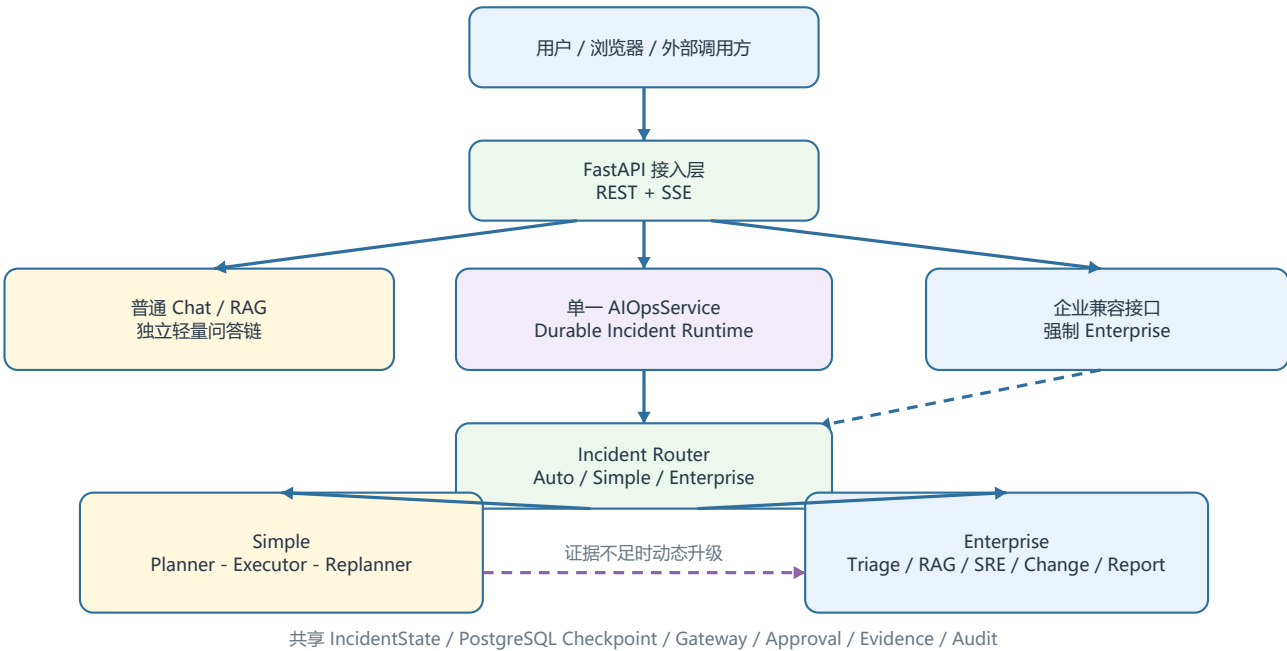


图 1 简化系统总览：普通 Chat 独立；/api/aiops 与企业兼容入口汇入同一个 AIOpsService，Router 决定策略。

路径 / 策略	适合做什么	状态位置	治理强度
普通 Chat / RAG	快速问答、知识检索、只读辅助查询	后端 MemorySaver + 浏览器 localStorage	轻量工具链，与事故执行隔离
Simple	单服务、证据路径较清晰的逐步诊断	统一 IncidentState + PostgreSQL	Gateway、Policy、审批、Evidence、Audit
Enterprise	跨服务、日志、指标、变更、图谱的复杂诊断	同一 IncidentState + PostgreSQL	共享安全底座并隔离 Provider 失败

1.1 建立四层心智模型

- 接入层：FastAPI 负责 HTTP、参数校验、身份信息、JSON 和 SSE。
- 运行时层：AIOpsService 持有唯一事故父图，负责路由、执行、暂停与恢复。
- 策略与平台层：Simple / Enterprise 节点使用 Gateway、检索、GraphRAG 和 Evidence。
- 数据与外部依赖：PostgreSQL、Milvus、本地文件、NetworkX、Qwen、MCP、Prometheus。

1.2 最值得讲的工程点

- 确定性路由：安全关键路径由规则决定，显式策略始终覆盖自动判断。
- 动态升级：Simple 证据不足时保留已完成结果，最多升级一次到 Enterprise。
- 持久化父图：两种策略的每个节点共享 PostgreSQL Checkpointer。
- 工具治理：身份、风险、Policy、审批、超时、重试、dry-run 和审计统一收口。
- Provider 失败隔离：单个外部数据源失败不会伪造结果，也不会阻断其余角色。

- 向后兼容：保留旧节点名，v1 Checkpoint 依据 next node 恢复而不重新路由。

设计边界

普通 Chat 是独立轻量问答入口；生产事故、写操作、人工审批和跨进程恢复必须进入 Durable Incident Runtime。MCP 服务是演示适配器，真实地址、权限和数据范围由部署环境提供。

2. 一次 HTTP 请求如何进入 FastAPI

2.1 把 HTTP 想成带格式的网络快递单

请求包含方法、URL、请求头和请求体。FastAPI 根据方法与路径匹配路由，用 Pydantic 校验输入，再把业务工作交给 Service。

```
POST /api/aiops HTTP/1.1
Content-Type: application/json

{"incident_id":"inc-2026-001","input":"支付服务错误率升高","strategy":"auto","execute_remediation":false}
```

2.2 当前三个公开入口怎么分工

入口	行为	关键说明
POST /api/chat	普通问答 / RAG	不进入事故 Runtime
POST /api/aiops	Auto / Simple / Enterprise	默认 auto，进入统一 AIOpsService
POST /api/enterprise/incidents	兼容企业事故请求	内部强制 enterprise，仍调用同一 Service

2.3 路由层通常做四件事

- 接收请求：读取 JSON、Header、身份和事故字段。
- 解析依赖：取得 AIOpsService、连接资源和权限上下文。
- 调用业务层：不在 API 函数中实现诊断算法。
- 转换响应：返回 JSON，或把 Graph 更新转换成连续 SSE。

2.4 REST 与 SSE 的区别

方式	连接体验	适合场景
普通 REST	服务端完成后一次返回	聊天、状态查询、企业兼容 JSON 结果
SSE	同一 HTTP 连接持续发送事件	计划、路由、升级、Agent 更新、审批和完成状态

```
浏览器 / 外部调用方
-> FastAPI Router
-> AIOpsService / RagAgentService
-> Graph Node / Tool / Retrieval
-> PostgreSQL / Milvus / Provider
-> JSON 或 SSE
```

3. 普通聊天: Agent + RAG + MCP

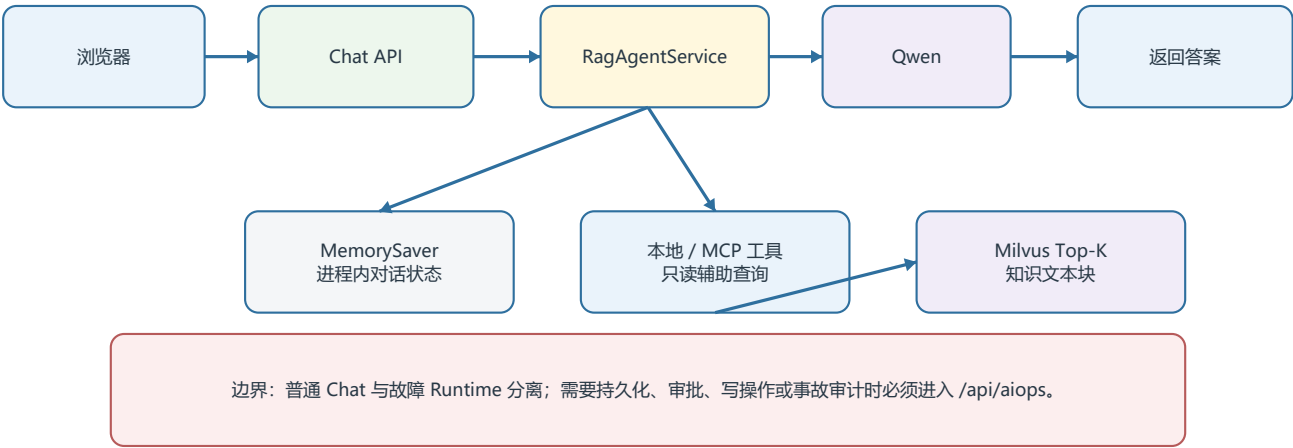


图 2 普通聊天调用链：轻量问答链与正式事故 Runtime 保持隔离。

3.1 用户问一句话，后端发生什么

- 浏览器调用 /api/chat 或 /api/chat_stream。
- Chat API 把请求交给 RagAgentService。
- Qwen 判断直接回答、知识检索或调用只读工具。
- Milvus 返回 Top-K 文本块，模型结合上下文生成答案。
- 普通接口一次返回；流式接口逐段返回。

3.2 RAG 解决什么问题

RAG 的三个步骤

Index：文档切分并写入向量库。Retrieve：问题到来时召回 Top-K。Generate：模型结合检索内容生成答案。

3.3 MemorySaver 与 localStorage 为什么同时存在

位置	保存什么	重启影响
后端 MemorySaver	当前进程的普通 Chat Agent 状态	后端进程重启后消失
浏览器 localStorage	前端会话列表和展示历史	保留在同一浏览器本地

3.4 MCP 的角色与边界

MCP 提供日志和监控工具协议。普通 Chat 只承担辅助查询；正式事故工具必须在 Runtime 中经 Tool Gateway 执行。这样可以避免一个聊天入口同时承担自由问答和生产处置两种不同风险模型。

4. 文档如何进入 Milvus 知识库

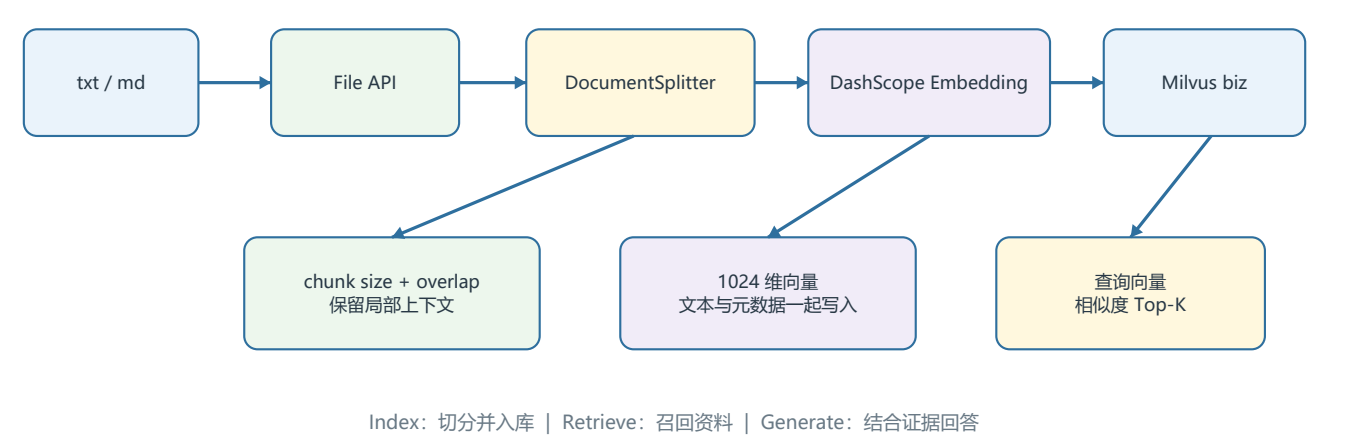


图 3 文档入库与检索：原始文本逐步变成可召回的 1024 维向量。

4.1 从上传到可检索的六步

- File API 校验 txt 或 md 文件并写入 uploads。
- DocumentSplitter 按 chunk size 和 overlap 切分文本。
- Embedding 服务调用 DashScope 生成 1024 维向量。
- VectorIndexService 把向量、原文和元数据写入 biz collection。
- 查询时 VectorSearchService 生成问题向量并召回 Top-K。
- 检索证据返回 Agent，参与最终生成。

4.2 为什么要切 chunk

整篇长文档只生成一个向量会混合多个主题。切块后可以精准召回具体步骤；overlap 能减少一句话被边界截断造成的语义损失。

4.3 关键代码位置

职责	主要代码
文件上传	app/api/file.py
文档切分	app/services/document_splitter_service.py
向量化	app/services/vector_embedding_service.py
写索引	app/services/vector_index_service.py
Milvus 管理	app/services/vector_store_manager.py; app/core/milvus_client.py
向量查询	app/services/vector_search_service.py

5. 项目核心：单一 Durable Incident Runtime

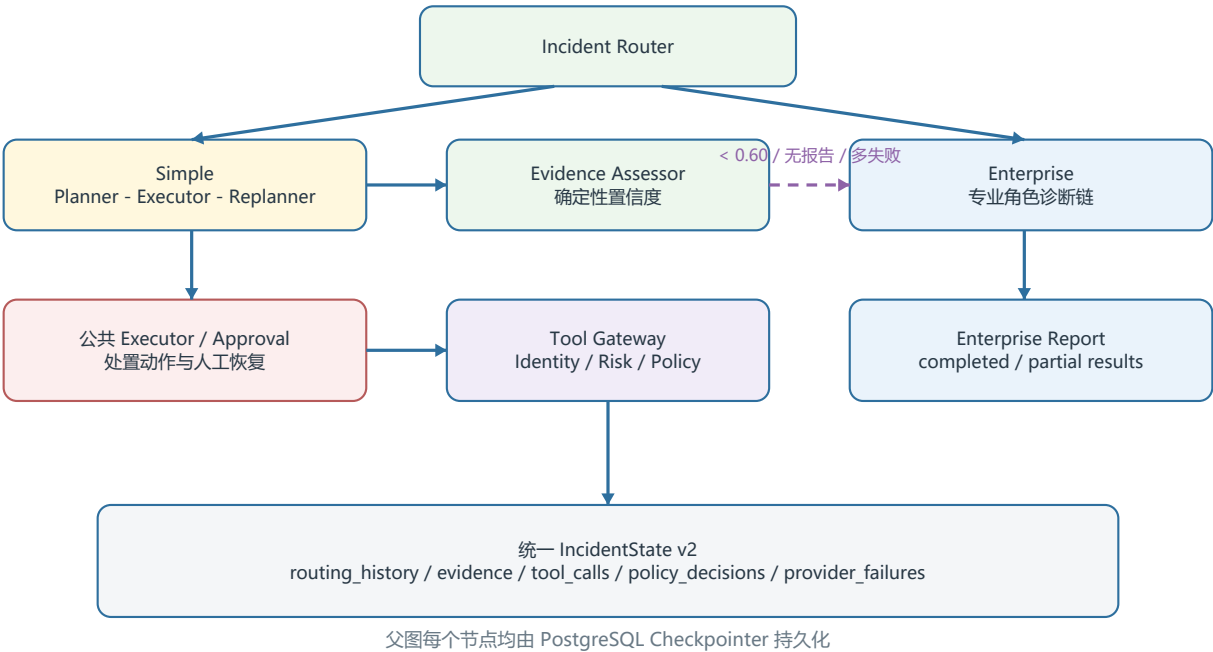


图 4 单一父图：Router 选择策略，Simple 可动态升级，Enterprise 与公共 Executor / Approval 共享同一状态和 Checkpoint。

5.1 为什么诊断流程要做成 Graph

事故诊断包含计划、执行、证据判断、并行调查、人工审批和恢复。LangGraph 用节点、边和 State 表达这些步骤，使每个状态变化都有清晰边界并可持久化。

5.2 IncidentState v2 是统一事故工作台

字段	用途
workflow_version	新请求为 2；缺失时按 v1 Simple 状态兼容读取
requested_strategy	调用方请求 auto / simple / enterprise
selected_strategy	Router 实际选择 simple 或 enterprise
routing_history	记录初始路由、动态升级、原因码和时间
diagnosis_confidence	Simple 证据评分，范围 0 到 1
provider_failures	外部 Provider 失败及角色降级信息
evidence / tool_calls / policy_decisions	证据、工具和安全决策的可追溯链

5.3 Router 为什么不用 LLM 决定

策略选择影响成本、工具范围和执行路径，因此采用确定性规则。显式 simple / enterprise 始终覆盖 auto；auto 对 critical、多服务、明确 GraphRAG / 变更关联，或 high + recent_change 直接进入 Enterprise，其余先走 Simple。

5.4 Simple 如何动态升级

证据项	分值
命中 Runbook	+0.20
不同证据来源	每种 +0.15, 最高 +0.45
成功工具调用	1 次 +0.10; 2 次以上 +0.20
报告包含有效引用	+0.15
存在工具失败	-0.15
失败步骤至少 2 次	再 -0.15

升级条件

仅 auto 可以升级；置信度低于 0.60、没有报告或至少两个失败步骤时升级一次。显式 Simple 不升级，Enterprise 不降级。升级会保留 evidence、tool audit、policy decision 和 past steps，并清理未完成计划与旧 response。

5.5 Planner、Executor、Replanner 如何分工

- Planner：把告警拆成可执行诊断步骤。
- Executor：只推进当前步骤，通过 Gateway 取得真实证据。
- Replanner：根据已有证据继续、调整或生成报告。
- Evidence Assessor：用确定性公式评估 Simple 是否足够。
- Enterprise 节点：复杂故障下补充专业并行调查。

```
START -> incident_router
-> simple -> planner -> executor -> replanner -> evidence_assessor
-> complete / enterprise_triage
-> enterprise -> triage -> rag -> sre + change -> root cause -> remediation -> report
```

6. Tool Gateway：为什么企业 Agent 必须管住工具

6.1 直接让 LLM 调生产工具有什么风险

模型不应该直接拥有无限工具权限。日志、指标、变更和写操作必须回答：谁在调用、能访问哪个服务、风险多高、是否需要审批、执行是否超时、结果能否审计。

6.2 Gateway 的执行漏斗

- Agent Node
- > 创建请求级 Gateway 实例
 - > Identity Scope
 - > Risk Metadata
 - > Policy-as-Code
 - > Allow / Deny / Require Approval
 - > Timeout + Retry + dry-run
 - > Audit + Evidence

6.3 为什么不能使用可变全局 audit hook

多个 Incident 并发时，全局可变 hook 可能把事故 A 的工具审计写到事故 B。项目共享工具注册、风险和策略实现，但通过 factory 为每次节点调用创建隔离 Gateway，保证审计上下文不串线。

6.4 restart_service 为什么仍然安全

控制点	行为
风险等级	write
默认模式	execute_remediation=false，只生成建议
身份	Observer 无权执行；Operator 才能进入审批
production	必须审批
执行层	永远强制 dry_run=true，不真实重启服务
审计	工具调用、Policy、审批决定和 Evidence 同一链路可追溯

容易混淆

“Gateway 允许执行” 不等于 “动作一定真实落地”。工具仍可声明 dry-run 约束；restart_service 的实现拒绝 dry_run=false。

7. 路由、升级、审批、Checkpoint 与恢复

7.1 Checkpoint 保存什么

Checkpoint 是父图运行到某个节点后的可恢复快照。Simple 与 Enterprise 都在同一个 Graph 中，因此路由历史、Agent 输出、Provider 失败、审批状态和最终报告都能按 incident_id 保存。

7.2 一次审批暂停的时间线

- Executor 生成需要审批的工具调用。
- 流程进入保留名称的 approval 节点。
- LangGraph interrupt 暂停，PostgreSQL 已保存当前状态。
- 前端显示 approval_required 卡片；Observer 不能越权。
- Operator 通过审批 API 批准或拒绝。
- 后端以 Command resume 恢复同一 thread；已完成 Agent 不重复执行。
- 批准后只执行受控 dry-run；拒绝结果进入审计并继续形成报告。

7.3 incident_id、thread_id 与 trace_id

字段	生命周期	用途
incident_id	贯穿一场事故	业务事故标识
thread_id	贯穿 LangGraph 状态	当前等于 incident_id，用于找 Checkpoint
trace_id	贯穿一次具体运行	串联本次日志、事件和调用

7.4 旧 Checkpoint 如何兼容

缺失 workflow_version 的状态按 v1 Simple 读取；planner、executor、replanner、approval 节点名继续保留。恢复旧线程时遵循 Checkpoint 保存的 next node，不重新经过新 Router。

7.5 恢复不等于外部副作用自动 exactly-once

工程事实

Checkpoint 保证 workflow 状态可恢复。真实外部副作用还需依赖幂等键、操作审计和业务去重。本项目的 restart_service 强制 dry-run，从源头避免演示环境产生真实重启副作用。

8. Enterprise 多智能体诊断策略

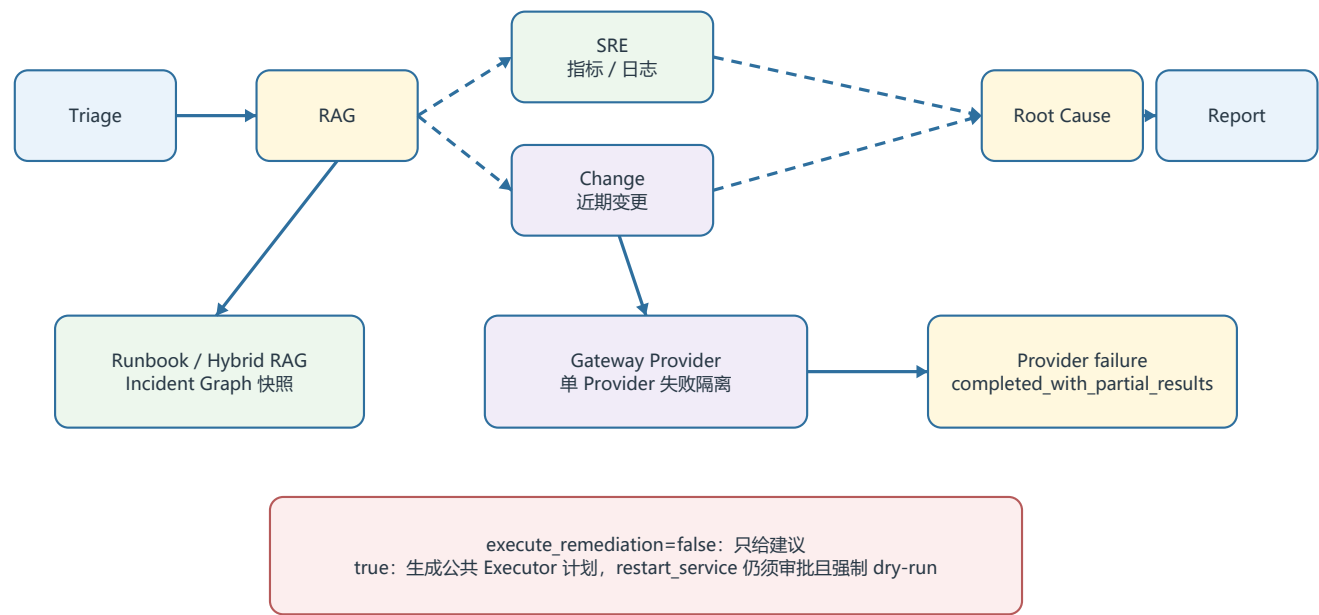


图 5 Enterprise 是统一父图中的专业策略：固定角色、并行调查、Provider 失败隔离，并可进入公共执行与审批链。

8.1 多 Agent 在这里怎么理解

这里不是多个模型自由讨论，而是固定角色和固定拓扑。每个节点输入输出结构明确，便于复现、评测、持久化和审计。EnterpriseIncidentWorkflow 保留类名，但生产 API 不独立运行它；它向 AIOpsService 父图提供节点。

8.2 每个角色负责什么

角色	职责	外部能力
Triage	判断优先级、影响范围和服务集合	事故输入
RAG	检索 Runbook、知识、历史事故和图谱	Hybrid RAG + Incident Graph
SRE	分析指标、日志与服务关系	Gateway -> MCP / Prometheus
Change	关联近期发布和配置变更	Gateway -> 变更 Provider
Root Cause	汇合证据并给出根因置信度	Evidence
Remediation	匹配 Runbook 并生成建议或执行计划	公共 Executor / Approval
Report	生成结构化事故报告	统一 IncidentState

8.3 为什么 SRE 与 Change 可以并行

指标 / 日志调查与近期变更关联是两条相对独立的证据链。父图并行执行后在 Root Cause 汇合，不依赖 Kafka 或 RabbitMQ；并行状态仍由 LangGraph 和 Checkpointer 管理。

8.4 Provider 失败时为什么不能伪造结果

外部 MCP 或变更 Provider 不可用时，相应角色标记 failed，并写入 provider_failures。其他角色继续工作，最终状态为 completed_with_partial_results。报告明确缺失数据，而不是用硬编码 CPU、日志或临时图冒充真实证据。

8.5 Hybrid RAG 与 GraphRAG 的区别

能力	回答的问题	当前实现
Hybrid RAG	哪些文本资料最相关	关键词与语义融合
GraphRAG	服务、依赖、变更和历史事故如何关联	启动时加载 NetworkX Incident Graph 快照

8.6 状态、来源与完成结果如何统一

对象	统一记录
角色输出	role、status、summary、evidence_refs 和耗时
样例图数据	明确 source=sample provenance，不与真实 Provider 证据混淆
外部失败	provider_failures 保存 Provider、角色、错误与时间
最终状态	completed 或 completed_with_partial_results

8.7 企业兼容接口为什么仍然保留

/api/enterprise/incidents 保留旧调用方式并强制选择 Enterprise，便于既有客户端迁移；内部不再创建第二个 Workflow，而是转发到同一个 AIOpsService，因此不会产生两套 Checkpoint、Gateway 或审计口径。

9. 启动、存储与运行边界

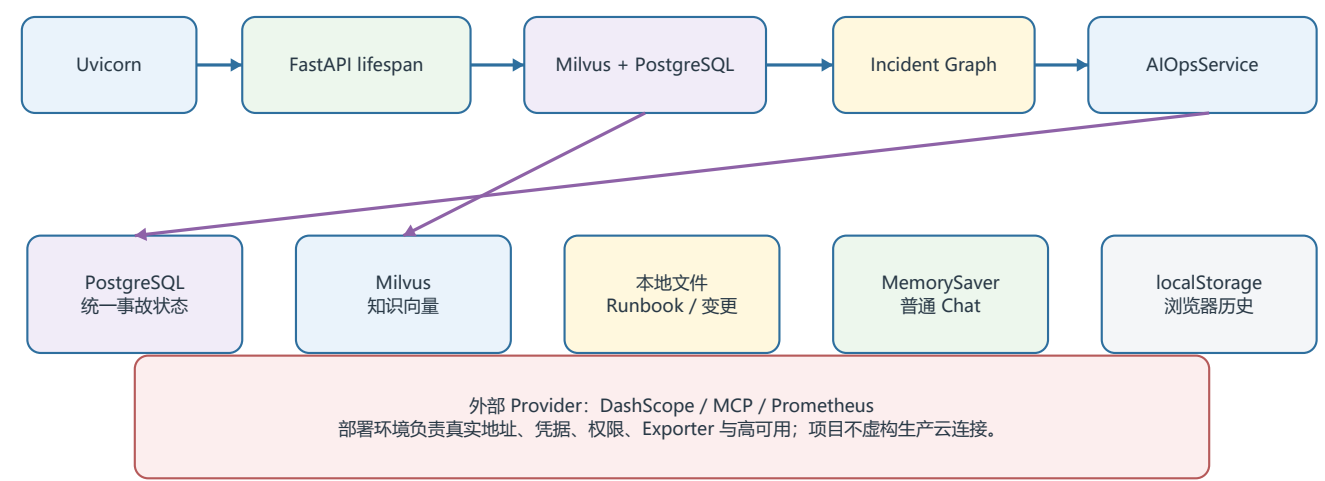


图 6 启动与存储：单一 AIOpsService 使用 PostgreSQL；普通 Chat 状态与浏览器历史保持独立。

9.1 应用启动顺序

```
app.run / Makefile / start-windows.bat
-> Uvicorn
-> FastAPI lifespan
-> Milvus + PostgreSQL Saver
-> Incident Graph 快照
-> Enterprise 节点提供者
-> 单一 AIOpsService
-> 监听 9900
```

9.2 五种数据位置

位置	保存内容	重启后
PostgreSQL	两种事故策略的统一 LangGraph Checkpoint	保留
Milvus	知识文档向量	保留
本地文件	上传文档、Runbook、变更、图快照、评测数据	取决于磁盘 / Volume
进程内内存	普通 Chat MemorySaver、加载后的 NetworkX 图对象	重启后按来源重新加载
浏览器 localStorage	前端会话列表	保留在该浏览器本地

9.3 外部依赖和默认端口

依赖	用途	边界
DashScope / Qwen	对话、规划、推理、Embedding	需要有效模型配置
MCP Server	日志、监控工具协议适配	演示适配器；真实权限由部署环境提供

依赖	用途	边界
Prometheus	指标查询	默认 9090，可配置
PostgreSQL	LangGraph Checkpoint	备份和高可用属于数据库基础设施
FastAPI	主服务	默认 9900

运行边界

项目提供本地与 Compose 运行方式，不宣称已经连接真实生产云资源。OpenTelemetry / AgentOps 的 Exporter、生产 MCP 地址、凭据和权限均由部署环境注入。

9.4 启动时对象之间是什么关系

- Checkpoint Runtime 创建 PostgreSQL Saver，并注入 AIOpsService。
- EnterpriseIncidentWorkflow 提供节点、角色运行器和 GraphRAG 能力，不独立编译生产图。
- Incident Graph 在启动时从可追溯快照加载为 NetworkX 对象，供 Enterprise RAG 使用。
- Gateway factory 在节点调用时创建隔离实例，避免跨 Incident 串审计。

9.5 Provider 不可用时 Runtime 为什么还能完成

PostgreSQL 和父图属于运行时基础；单个 MCP、Prometheus 或变更 Provider 属于证据来源。证据来源失败会降低结果完整性并形成 partial results，但不会用演示数据填补，也不会抹掉其他角色已经取得的有效证据。

10. 代码目录应该按什么顺序读

沿请求的真实流向阅读，比按目录逐个文件看更容易建立运行时地图。下面顺序能覆盖从 HTTP 到 Checkpoint 的完整链路。

10.1 推荐的十步源码阅读顺序

顺序	位置	重点
1	app/main.py	路由、lifespan、资源和单一 AIOpsService 初始化
2	app/api/aiops.py	统一事故入口、企业兼容入口、状态与审批 API
3	app/models/aiops.py	strategy、input、identity、execute_remediation
4	app/services/aiops_service.py	唯一 StateGraph 的节点、边、SSE 和恢复
5	app/agent/aiops/router.py	初始路由、证据评分和动态升级
6	app/agent/aiops/	Planner、Executor、Replanner、State 和 Approval
7	app/agent/tool_gateway.py	身份、风险、Policy、审批、审计和隔离 factory
8	app/agent/enterprise_workflow.py	企业节点、并行汇合、Provider 降级和报告
9	app/core/checkpoint.py	PostgreSQL Saver 和连接池
10	static/、tests/、docs/learning/	策略面板、SSE 展示、验收证据和学习材料

10.2 读一个节点时固定问六个问题

- 谁调用它，进入它的条件是什么？
- 读取和修改 IncidentState 的哪些字段？
- 是否调用模型、Gateway、数据库或 Provider？
- 失败如何写入状态，是否允许其余节点继续？
- 节点结束后 Checkpoint 保存在哪里？
- 下一条边如何选择，恢复后是否会重复执行？

实用阅读卡片

把每个函数记成 “输入 -> 主要动作 -> 外部依赖 -> State 更新 -> 输出 / 下一节点” 。读完后你会得到一张真实运行时地图。

11. 测试、调试和常见故障定位

11.1 当前验收基线

门禁	结果
全量 pytest	132 passed
代码覆盖率	66.42%
PostgreSQL integration	2 passed
Ruff	All checks passed
Pyright	0 errors
Python / JavaScript 语法	通过
PDF / 浏览器视觉检查	无乱码、裁切、重叠；策略与审批界面可复现

11.2 关键场景怎么验证

场景	必须观察的证据
Simple	Router 选 simple; plan / step / report 连续; 置信度可追踪
直接 Enterprise	critical / 多服务规则命中; Agent 节点并行并汇合
动态升级	routing_history 记录 simple -> enterprise; 既有 Evidence 保留
审批恢复	approval_required; 数据库重建 Service 后恢复; 已完成 Agent 不重复
Provider 失败	failed 角色写 provider_failures; 最终 partial results; 无伪造证据
v1 恢复	不重新路由; 按旧 next node 继续

11.3 按症状定位

症状	优先检查
服务起不来	PostgreSQL / Milvus、环境变量、lifespan 日志
策略选错	requested_strategy、severity、services、recent_change、routing_history
Simple 没升级	diagnosis_confidence、报告、失败步骤、escalation_count
Enterprise 数据缺失	provider_failures、Gateway audit、MCP / 变更 Provider
审批后不恢复	incident_id / thread_id、Checkpoint next、身份和 Command resume
恢复后重复执行	旧 Checkpoint 的 next node、节点副作用幂等和已完成 Agent 标记
SSE 前端不更新	event type、sequence、Content-Type、代理缓冲和浏览器网络面板

11.4 如何证明持久化是真的

让流程停在企业审批节点，关闭原 Service 和连接资源，再创建新的 AIOpsService，用同一个 incident / thread 恢复。验证审批状态仍在、已完成 Agent 不重复、最终审计链完整。这比只看数据库表里有行更有说服力。

12. 面试时怎么把项目讲清楚

12.1 30 秒版本

项目概述

这是一个面向企业 OnCall / SRE 的 Incident Response Agent 平台。系统用 FastAPI 提供普通 Chat、知识入库和统一事故接口；用一张持久化 LangGraph 承载 Simple 与 Enterprise 两种诊断策略，通过确定性 Router 自动选路并允许 Simple 在证据不足时动态升级；两种策略共享 PostgreSQL Checkpointer、Tool Gateway、审批、Evidence 和 Audit。知识侧使用 Milvus + Embedding，外部日志和指标通过 MCP / Prometheus 适配。

12.2 两分钟版本：按问题 -> 设计 -> 价值 -> 边界

- 问题：单次 Agent 回答无法覆盖长流程、复杂证据、工具权限、人工审批和服务重启。
- 统一运行时：AIOpsService 只编译一张 Graph，IncidentState v2 保存路由、证据、工具、审批和 Agent 输出。
- 双策略：Simple 控制成本；Enterprise 处理跨服务、变更和图谱问题；Router 规则可解释。
- 动态升级：Evidence Assessor 用确定性分数判断证据是否充分，最多升级一次且不丢历史。
- 安全执行：Gateway 统一身份、风险、Policy、审批、dry-run、超时、重试和审计。
- 持久化：PostgreSQL Checkpointer 使 Simple、Enterprise 和审批都能跨进程恢复。
- 失败隔离：单 Provider 失败形成 partial results，不伪造生产证据。
- 运行边界：普通 Chat 独立；MCP 是部署适配器；真实云凭据和生产高可用由环境提供。

12.3 高频追问与回答框架

追问	回答要点
为什么只编译一张 Graph？	统一 Checkpoint、审批、审计和 State，避免两条事故链状态漂移。
为什么 Router 不交给 LLM？	策略选择涉及成本和安全，确定性规则可解释、可测试、可审计。
为什么不是所有事故都用多 Agent？	简单事故用多角色会增加延迟和成本；Simple 先行并允许证据不足时升级。
升级为什么不会重复查一遍？	升级保留 Evidence、tool audit、policy decision 和 past steps，并清理未完成计划。
为什么需要 PostgreSQL？	审批等待和服务重启要求状态跨进程保留；内存状态无法满足。
Tool Gateway 的价值？	把模型和真实工具隔开，统一权限、风险、Policy、审批和审计。
Provider 挂了怎么办？	失败写入 provider_failures，其余角色继续，报告明确 partial results。
GraphRAG 与向量 RAG 区别？	向量 RAG 找相似文本；GraphRAG 扩展服务、变更和历史事故关系。
恢复是否 exactly-once？	Graph 状态可恢复；真实外部副作用还需工具幂等和审计。
为什么 restart_service 安全？	production 必须审批，执行层始终强制 dry-run，不真实重启。

12.4 一句话总结项目亮点

一句话

这个项目把 Agent 从一次性问答推进为可自动选路、可动态升级、可恢复、可审批、可审计的事故处理 Runtime，同时保留 RAG、多角色并行分析和受控处置能力。

12.5 关键设计取舍怎么回答

取舍	为什么这样设计
Simple 优先而非全量多 Agent	控制延迟和模型成本，用动态升级覆盖复杂故障
规则 Router 而非 LLM Router	安全关键决策可解释、可复现、可单元测试
单一父图而非两个服务	统一持久化、审批、工具策略和审计语义
Provider 失败返回 partial	宁可显式缺失，也不生成无法追溯的生产事实
restart_service 只 dry-run	在演示工程中展示审批链，而不制造真实生产副作用

12.6 简历描述的推荐结构

- 场景：面向 OnCall / SRE 的长流程事故诊断。
- 核心：单一 LangGraph Runtime + 确定性双策略路由 + 动态升级。
- 可靠性：PostgreSQL Checkpoint、interrupt / resume、旧状态兼容。
- 安全性：Gateway、Policy、审批、dry-run 和端到端审计。
- 效果：132 项测试通过，覆盖 Simple、Enterprise、升级、Provider 降级和审批恢复。

附录 A. 关键术语速查

术语	一句话理解
FastAPI	负责 HTTP 路由、参数校验、依赖注入和响应。
SSE	服务端通过一个 HTTP 连接持续发送事件。
LangGraph	用节点、边和 State 编排有状态 Agent 工作流。
IncidentState	统一事故工作台，保存路由、证据、工具、审批和报告。
Incident Router	根据显式策略和复杂度规则选择 Simple / Enterprise。
Simple	Planner - Executor - Replanner 的低成本诊断策略。
Enterprise	固定角色、并行调查、GraphRAG 和结构化报告策略。
Evidence Assessor	用确定性公式计算证据置信度并决定是否升级。
Checkpoint	父图状态快照，用于中断、审批和重启后恢复。
Tool Gateway	正式事故工具调用的统一治理边界。
Policy-as-Code	把允许、拒绝和审批规则写成可执行策略。
dry-run	模拟执行，降低外部副作用风险。
Evidence	带来源、内容、时间和引用的事故证据。
Provider failure	外部数据源失败记录，不用伪造结果填补。
partial results	部分 Provider 不可用时仍完成其余分析的状态。
RAG	先检索资料，再结合资料生成答案。
Embedding	把文本映射成向量，用于语义检索。
Milvus	保存知识文本向量的向量数据库。
MCP	连接外部工具和数据 Provider 的协议接口。
GraphRAG	利用服务、变更和事故关系扩展上下文。
NetworkX	当前用于加载 Incident Graph 快照的 Python 图结构。
incident_id	长期事故标识，同时作为 LangGraph thread_id。
trace_id	一次具体运行链路的追踪标识。

附录 B. 配套架构图索引与自测

教程中的六张学习图用于快速建立心智模型；需要核对完整节点、数据流和运行边界时，再查看流程图集合中的 2.1-2.5 与《完整项目pdf版》。

原始图	重点	对应教程
2.1 系统总览	Chat 独立、单一 Runtime、双策略与共享底座	第 1、2、5 章
2.2 普通聊天与知识入库	Chat API、MemorySaver、MCP、Milvus	第 3、4 章
2.3 持久化 AIOps	Router、Simple、升级、Gateway、Approval、Checkpoint	第 5-7 章
2.4 Enterprise	角色并行、Provider 隔离、Root Cause、Remediation	第 8 章
2.5 启动与存储	lifespan、数据库、图快照、外部依赖	第 9、11 章

真正掌握的标准

你应当能从一个具体请求出发，沿 API、Service、Router、Graph Node、Gateway 和 Storage 讲完数据流；同时说明为什么选择某种策略、何时升级、怎样审批、怎样恢复，以及为什么 Provider 失败不会变成伪造证据。

自测清单

- /api/aiops 如何从 FastAPI 走到 Incident Router？
- critical、多服务和 high + recent_change 如何选路？
- Simple 的置信度如何计算，什么时候升级？
- 升级后哪些 State 保留，为什么不会重复成功节点？
- SRE 与 Change 为什么可以并行，在哪里汇合？
- Provider 失败如何记录，最终状态是什么？
- restart_service 为什么必须审批且不会真实重启？
- 旧 v1 Checkpoint 如何恢复而不重新路由？
- Milvus、PostgreSQL、MemorySaver、NetworkX 各保存什么？
- 如何用 30 秒和 2 分钟两种版本讲清项目？

学习入口

从 app/main.py 开始，按第 10 章顺序阅读；每看完一个节点，就把它映射回图 4 或图 5，并核对 State 更新和下一条边。